

UEH CREATIVE ROBOT CONTEST 2026

Simulation Round Technical Report

Autonomous Vision-Based Navigation, Perception, Behaviour Arbitration and Recovery
for TurtleBot3 Waffle

Participant	Nguyen Tuan Kiet
School	University of Economics Ho Chi Minh City (UEH)
Faculty	Institute of Intelligent and Interactive Technologies
Primary launch command	ros2 run crc_sim starter

Controller version: V4.8

ROS 2 Humble | Gazebo Classic 11 | OpenCV / NumPy | Camera + LiDAR + Odometry + IMU

Technical report prepared for the UEH CRC 2026 Simulation Round

Executive Summary

This report presents the final V4.8 autonomous navigation stack for the UEH Creative Robot Contest 2026 simulation round. The controller is built around a connected right-boundary polyline rather than a single detected line. White road markings are extracted from the forward camera, filtered by adaptive thickness and geometric consistency, connected into an ordered chain, and sampled at multiple look-ahead heights. Steering combines lateral error, path heading and delayed-preview curvature information. LiDAR provides front-range safety, while odometry and IMU yaw support lane-loss recovery and the structured highway obstacle-bypass manoeuvre.

The same robot camera is processed by a separate low-rate perception pipeline for STOP, RAMP, UNEVEN, highway entry/exit, BUS, TUNNEL and CROSSWALK signs, as well as traffic-light states and the simulator pedestrian. In the final code, several of these observations are coupled directly to motion: confirmed STOP signs freeze the controller for a timed hold; RED/YELLOW traffic lights stop the robot and GREEN releases it; RAMP signs trigger a temporary speed boost; a confirmed CROSSWALK starts a delayed camera-plus-LiDAR pedestrian-yield sequence; and HW_ENTRY/HW_EXIT arm and disarm the obstacle-bypass policy. BUS and TUNNEL remain detection-only informational classes.

Representative full-loop development runs cover approximately 28.3 m of cumulative odometry distance. The integrated controller uses a lower mean forward command and substantially more steering activity than the baseline trace. This is treated as an engineering trade-off: the final node deliberately spends time in confirmation windows, conservative stops, lane-loss recovery and safety states, while also using brief state-dependent boosts when RAMP, GREEN-light or crosswalk-clear conditions are satisfied. Completion time is therefore reported descriptively rather than treated as the primary optimization objective.

Contents

1. Approach and System Architecture
 2. Perception
 3. Control, Behaviour Arbitration and Safety
 4. Experimental Methodology
 5. Results and Discussion
 6. Limitations and Future Work
 7. AI Usage Statement
 8. Conclusion
- References

1. Approach and System Architecture

1.1 Design objectives

The track requires one continuous autonomous run through narrow lanes, intersections, a ramp, a dark tunnel, a pedestrian crossing and a highway section containing a stationary robot. The controller was therefore designed around three priorities: maintain a usable lane reference under changing geometry and illumination; keep safety behaviour independent from fragile visual detections; and recover from temporary loss of the lane without relying on prohibited simulator ground-truth information.

The solution uses only the robot-facing interfaces allowed for the graded controller: the forward camera, 2-D LiDAR, wheel odometry and IMU. The controller publishes velocity commands to `/cmd_vel` and exposes a `/lane_debug/image` topic for development visualization. No external trained model weights are required, which keeps deployment inside the provided ROS 2 environment simple and reproducible.

1.2 End-to-end pipeline

At runtime, the motion controller executes at 20 Hz and the camera perception pipeline at 5 Hz. The forward image is copied into two independent paths: a lane-control path that produces the white-marking mask and connected boundary estimate, and a detection path that classifies roadside signs, illuminated traffic-light lamps and (only when the crosswalk state machine requests it) the pedestrian. LiDAR is processed independently before lane/lost behaviour so close-range safety can override motion. Odometry and IMU provide travelled distance, pitch and yaw for recovery, ramp awareness and the rectangular bypass manoeuvre. Behaviour arbitration then selects between STOP/traffic-light holds, crosswalk yield logic, default obstacle stop/wait, highway bypass, lost-lane recovery and normal lane tracking before publishing `/cmd_vel`.

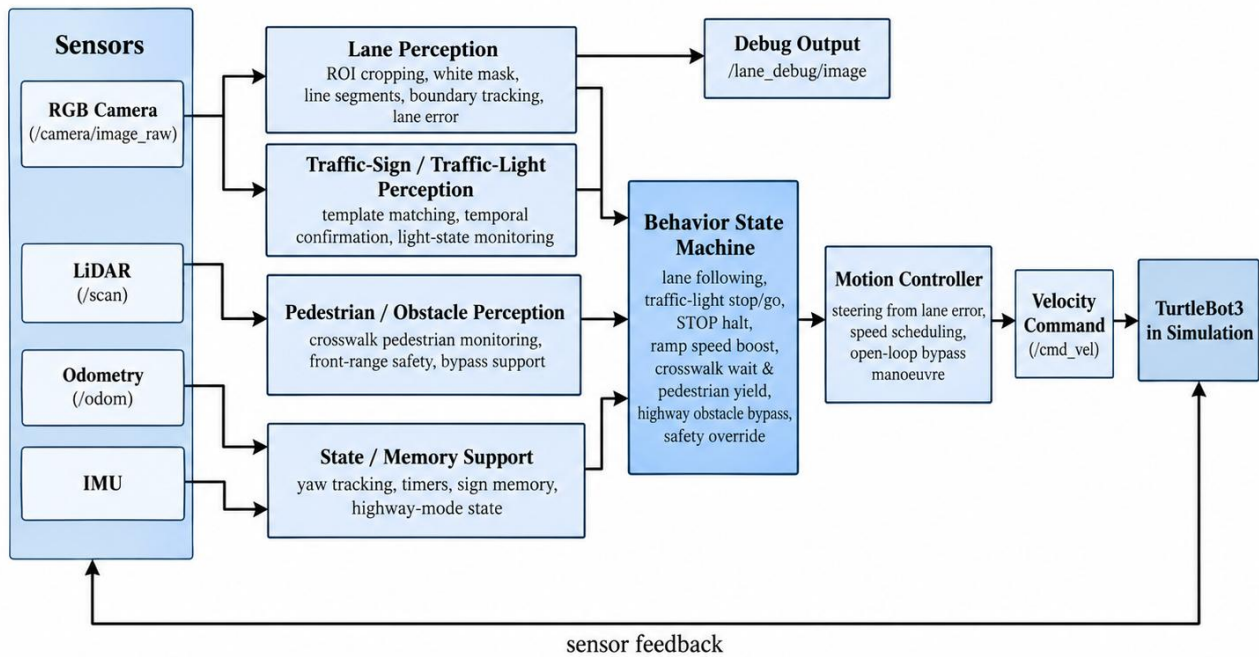


Figure 1. Overall system architecture showing the separate lane-control and perception pipelines, LiDAR safety, IMU/odometry recovery, behaviour arbitration, and velocity-command output. BUS and TUNNEL remain detection-only classes.

1.3 Main modules

Table 1. Functional modules in the submitted controller.

Module	Primary input	Main function	Output / role
Lane perception	Camera	White mask, Hough vectors, adaptive thickness, connected right-boundary chain	Near/mid/look/far path samples
Lane tracking	Lane samples	Temporal plausibility, smoothing and preview reference	Stable active-boundary estimate
Recovery	Camera + IMU/odom	3 s yaw hold, cardinal-anchor recovery, persistent SEEK BOTH, left fallback, escape turn	Reacquired boundary or controlled creep
Front safety	LiDAR + IMU pitch	Front-sector percentile, multi-frame confirmation, ramp-aware threshold	STOP/WAIT or behaviour-specific hand-off
Sign/light perception	Camera	HSV candidates, geometric/symbol heuristics, temporal confirmations	STOP/RAMP/HW/CROSSWALK/light states; BUS/TUNNEL detect-only
Crosswalk pedestrian gate	Camera + LiDAR	Delayed pedestrian watch after CROSSWALK; sensor agreement	STOP while person+obstacle agree; clear boost
Highway bypass	HW state + LiDAR + odom/IMU	Rectangular detour; detection paused during manoeuvre	Obstacle pass then fresh right-boundary reacquisition
Behaviour arbitration	All above	Prioritises STOP/light holds, crosswalk, obstacle policy, recovery, lane control	/cmd_vel

1.4 Why a connected boundary representation

A single-line lane follower is fragile on bends, dashed markings and partial occlusion. The submitted approach instead constructs a connected polyline from multiple compatible line vectors. This provides two advantages. First, the near sample gives a local lateral reference, while farther samples encode the direction of the road before the robot reaches the curve. Second, chain continuity provides a natural geometric check against isolated white structures such as crosswalk stripes or other transverse markings. The result is a compact representation that is still lightweight enough for real-time CPU execution.

2. Perception

2.1 Lane-marking extraction

Lane extraction starts with an HSV-based white mask. The default mask keeps bright, low-saturation pixels, then applies a small morphological opening to suppress isolated noise. Only the lower region of the image is considered for lane tracking, which reduces interference from signs, sky and distant scenery. Probabilistic Hough segments are generated from this mask. Candidate segments are screened using vertical span, orientation and extrapolation constraints before entering the lane model.

A distinguishing element of V4.8 is adaptive lane thickness. The controller measures the width of white markings perpendicular to each candidate segment at several points and maintains a reference thickness over time. Candidates that are too thin, too thick, or inconsistent with the current reference are rejected. This is useful near crosswalks, intersections and other regions where white structures can otherwise dominate a simple Hough detector.

2.2 Temporal tracking and transverse-mark rejection

Initial lane acquisition requires several mutually consistent measurements rather than immediately accepting the first frame. Once locked, the controller limits sudden jumps in near-lane position and path angle, then exponentially smooths the path samples and reference heading. A separate transverse detector checks segments whose angle differs strongly from the current lane direction. These detections are held for several frames and treated as a cue that a crosswalk or intersection marking may be present rather than as a new longitudinal lane boundary.

2.3 Traffic-sign perception

The sign detector is intentionally lightweight and simulator-specific. Red, blue and green colour masks generate candidate contours. Candidates are screened by area, aspect ratio and colour occupancy, classified, and then passed through class-agnostic non-maximum suppression. Only a sufficiently large, high-confidence roadside sign in the right-side image region is retained for each encounter. A release counter prevents the same physical sign from creating repeated events frame after frame.

Blue information signs are separated from the white symbol inside the plate. A large triangular white component is used for the pedestrian-crossing sign. BUS and TUNNEL are then distinguished from a deeper central crop using component structure, horizontal strength, enclosed dark holes and an arch-hollow ratio; ambiguous blue candidates are deliberately left unclassified rather than forced into a class. Green highway entry and exit signs are separated using the red diagonal slash on the exit plate. Red signs first use red occupancy and polygon structure to identify STOP; remaining warning signs use the complexity of the dark internal wave symbol to distinguish the RAMP/UNEVEN family. These heuristics avoid an external model dependency while keeping each decision interpretable in the debug output. BUS is exposed immediately once its geometry and size gates are satisfied, whereas TUNNEL must persist for 1.0 s before it is reported as a confirmed detection.

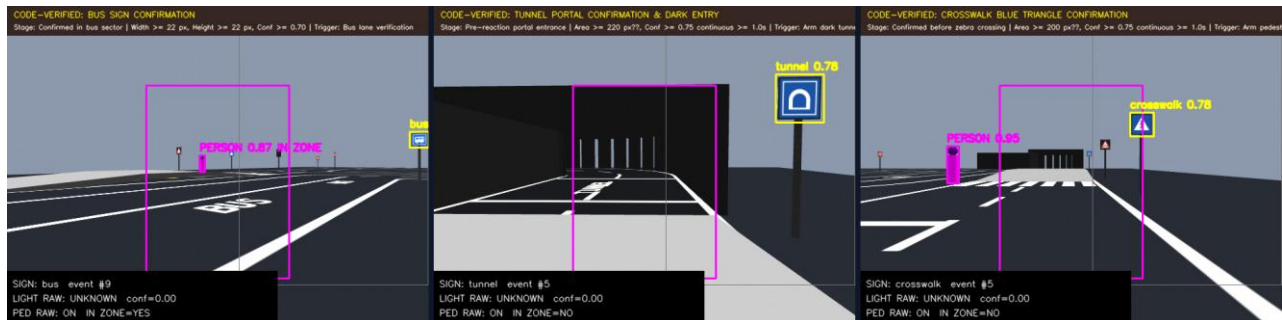


Figure 2. Representative blue-sign detections for bus stop, tunnel and pedestrian-crossing signs.

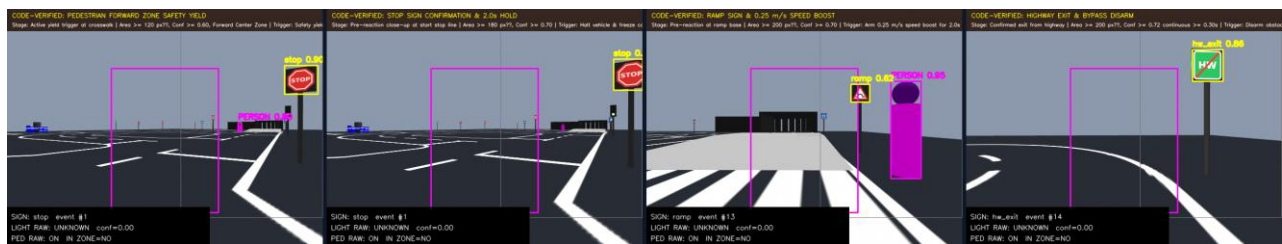


Figure 3. Representative pedestrian, STOP, RAMP and HW_EXIT detections. In the final controller, STOP, RAMP, CROSSWALK/pedestrian and highway-entry/exit observations are behaviour-coupled; BUS and TUNNEL remain detection-only.

2.4 Traffic-light and pedestrian observation

Traffic-light perception creates separate HSV masks for red, yellow and green illuminated pixels. Small contours are screened by area, aspect and circularity, and a dark-surround test favours compact lamp candidates inside a darker housing rather than large coloured signs. The detector reports the best RED, YELLOW, GREEN or UNKNOWN candidate with a confidence score. Behaviour logic accepts a lamp state only when the same colour remains above the configured confidence threshold on fresh perception frames for a continuous confirmation interval.

Pedestrian detection is not run continuously. A CROSSWALK sign must first pass its own temporal confirmation. After the confirmed sign disappears, the robot continues lane following for 3.0 s before enabling the magenta pedestrian detector. During the subsequent WATCH/BOOST phases, an upright magenta component

must lie inside the forward image zone, and an actual stop is commanded only when that fresh camera observation agrees with a close front-LiDAR return. When both sensors are clear, the controller briefly accelerates through the zebra to leave the crossing zone.



Figure 4. Traffic-light perception examples for red, yellow and green lamps. In the final code, RED/YELLOW are behaviour-coupled to a stop state after temporal confirmation, while confirmed GREEN releases the hold and can enable a temporary 0.20 m/s lane-follow speed while the green observation remains fresh.

3. Control, Behaviour Arbitration and Safety

3.1 Steering law

Normal lane control combines three errors. Lateral error compares the tracked near-boundary position with the learned near target. Heading error compares the current connected-path angle with the reference path angle. Preview error compares the near-to-look-ahead displacement with the corresponding displacement observed when the lane was acquired. The three terms are weighted, preview contribution is limited, and the resulting angular command is exponentially smoothed before being clamped to the configured turn limit.

This structure gives the controller both immediate centring and anticipatory curvature response. Lateral control prevents gradual drift, heading control aligns the robot with the local lane direction, and delayed preview begins steering before the near reference alone would demand a large correction. The smoothing stage suppresses abrupt frame-to-frame changes from noisy image geometry.

Table 2. Selected control and perception parameters from the submitted V4.8 controller.

Parameter	Default	Role
max_speed	0.10 m/s	Normal lane-follow upper forward command
max_turn	0.85 rad/s	Angular command limit
kp_lateral	0.58	Near-boundary lateral correction
kp_heading	0.16	Path-angle correction
kp_preview	0.30	Delayed-preview curvature correction
steer_alpha	0.24	Angular-command smoothing
stop_distance / ramp_stop_distance	0.30 / 0.30 m	Normal and ramp frontal safety distance in the final code
obstacle_confirm_frames	5	Reject one-frame LiDAR noise
stop_sign_confirm_sec / hold_sec	1.0 / 2.0 s	STOP temporal confirmation and frozen hold
traffic_light_min_conf / confirm_sec	0.86 / 0.8 s	Strict light-state temporal gate
ramp_boost_speed / sec	0.25 m/s / 2.0 s	Accepted RAMP speed override
crosswalk_confirm / post-sign delay	0.50 / 3.0 s	Crosswalk confirmation and delayed person watch
green / crosswalk boost speed	0.20 / 0.20 m/s	Temporary state-dependent lane-follow speeds
perception_rate_hz	5 Hz	Sign/light/pedestrian processing rate
highway_entry_confirm_sec	0.20 s	Continuous HW_ENTRY gate before bypass is armed

3.2 Speed scheduling and ramp handling

Normal forward speed is reduced as the absolute steering demand approaches the maximum turn limit, with a small minimum creep speed to avoid stalling. IMU pitch independently identifies ramp traversal and applies a conservative 0.060 m/s cap during ordinary ramp motion. A correctly accepted RAMP sign can temporarily override that cap with a 0.25 m/s boost for 2.0 s; confirmed fresh GREEN and crosswalk-clear states can likewise request 0.20 m/s. These higher state-dependent speeds are passed through an explicit per-command linear limit so the normal max_speed parameter does not clip the intended temporary boost.

3.3 Lost-lane recovery

Lane loss is handled as a staged recovery problem rather than an immediate stop. For approximately the first 3 s of a loss, the robot holds the yaw of the last valid lane while creeping slowly. Outside a persistent dual-look window, recovery then uses the nearest cardinal IMU heading as an anchor. After prolonged LONG LOST, the controller opens a timed SEEK BOTH window that searches both outer boundaries while continuing to preserve the most recent valid lane yaw instead of forcing the robot back to a cardinal heading.

If only the left boundary is recovered, it is used as a temporary proxy while the controller keeps searching for the preferred right boundary. The recovery logic also contains an escape case: if SEEK BOTH remains unable to establish a stable lock for 8 s, the robot performs a controlled 90-degree left rotation using IMU yaw and then continues a right-boundary-only search. A successful valid track re-arms the dual-look logic for a future loss episode. These states remain sensor-based and do not read simulator object positions or map coordinates.

3.4 LiDAR safety and obstacle bypass

The front LiDAR safety check operates independently from the lane camera pipeline. The controller takes a percentile of finite ranges in a narrow forward sector rather than trusting a single ray, and it confirms close obstacles across multiple control frames. By default, an obstacle produces STOP/WAIT behaviour. The structured detour is permitted only after a temporally confirmed HW_ENTRY and is disabled again by HW_EXIT; the crosswalk state machine has its own camera-plus-LiDAR ownership of the frontal obstacle decision.

When highway mode is active, the right boundary is locked and the robot is not on the ramp, the confirmed frontal obstacle can start a rectangular bypass. The state machine rotates left by 90 degrees, moves laterally out using odometry distance with yaw hold, rotates back to the original heading, passes the obstacle, rotates right, moves laterally back, and finally rotates left to resume the original heading. Camera sign/light/person perception is paused for the entire bypass, its temporal confirmation candidates are reset, and normal detection resumes only after the manoeuvre. The previous lane lock is then discarded and a fresh right-boundary acquisition is required.

3.5 Perception-coupled behaviour states

STOP-sign control uses only fresh detector frames. The same physical sign encounter must remain continuously visible for 1.0 s before acceptance, after which the robot immediately commands zero velocity and freezes the lane/lost/recovery controller state for a 2.0 s hold. When the hold expires, absolute recovery timers are shifted by the frozen duration so the pre-STOP controller state resumes consistently. An encounter identifier prevents the same physical STOP sign from triggering more than once.

Traffic-light control uses a stricter temporal gate: the same RED, YELLOW or GREEN state must remain above confidence 0.86 for 0.8 s on fresh perception frames. RED and YELLOW enter a motion hold; GREEN releases that hold and marks the signal interaction complete. While a confirmed green lamp remains fresh and the robot is not on the ramp, normal lane following may use the configured 0.20 m/s green speed. The implementation also includes a 10 s safety auto-release for an extended signal stop to avoid permanent deadlock during development testing.

The CROSSWALK state machine requires a confirmed crossing sign, waits until that sign disappears, follows the lane for 3.0 s, then enables pedestrian detection. A stop is issued only when a fresh magenta pedestrian lies inside the forward image zone and front LiDAR simultaneously reports an obstacle inside the active safety distance. Once clear, the controller applies a 1.5 s, 0.20 m/s lane-follow boost. Separately, a confirmed HW_ENTRY arms highway obstacle bypass, HW_EXIT restores default stop/wait behaviour, and a RAMP sign arms the temporary ramp-speed boost. BUS and TUNNEL are intentionally detected and reported without changing motion.

BEHAVIOUR COUPLING IN FINAL V4.8

Closed-loop: STOP -> timed controller freeze; RED/YELLOW -> stop; GREEN -> release / temporary fast lane-follow; RAMP -> timed 0.25 m/s boost; CROSSWALK -> delayed pedestrian camera watch + LiDAR corroboration -> stop/clear boost; HW_ENTRY/HW_EXIT -> arm/disarm rectangular obstacle bypass. Detection-only: BUS and TUNNEL. All temporal confirmations are based on fresh camera frames, and camera detection is paused during bypass.

4. Experimental Methodology

4.1 Simulation and recording setup

The simulator is the organizer-provided ROS 2 Humble and Gazebo Classic environment with a TurtleBot3 Waffle. The autonomous solution is started with the single command `ros2 run crc_sim starter` against an already-running simulation. The controller subscribes only to `/camera/image_raw`, `/scan`, `/odom` and `/imu` and publishes `/cmd_vel`. Two debug image topics are outputs only: `/lane_debug/image` for lane/controller state and `/detection_debug/image` for sign, light, pedestrian and behaviour-state diagnostics.

Participant-side evaluation logs subscribe to odometry and velocity-command data and record timestamp, elapsed time, odometry position and yaw, commanded linear and angular velocity, cumulative travelled distance and a moving-state flag. These logs are then used by the analysis scripts to produce trajectory, distance, speed and angular-velocity figures. This keeps quantitative evidence tied directly to recorded runs rather than manually transcribed observations.

4.2 Representative runs used in this report

The quantitative discussion uses two unique representative full-loop traces: the original baseline reference run and an updated integrated-controller run. Duplicate copies of the same raw trace are not treated as independent experiments. The aim of this comparison is not to establish a statistically complete benchmark; it is to show how the integrated controller changes motion behaviour while preserving full-course traversal in development testing.

Table 3. Representative recorded-run summary used for quantitative discussion.

Run	Elapsed time	Distance	Mean linear cmd	Mean angular cmd	Interpretation
Baseline Run 01	420.50 s	28.430 m	0.0990 m/s	0.0015 rad/s	Original full-loop reference trace
Updated Run 02	500.00 s	28.293 m	0.0586 m/s	0.0742 rad/s	Integrated controller with more active steering / stop behaviour

4.3 Evaluation interpretation

Elapsed time is reported as a descriptive systems metric rather than the main optimization target. The final controller intentionally lowers speed during stronger steering, ramp traversal, obstacle handling and recovery. As a result, a longer run can represent a deliberate stability/safety trade-off rather than a regression in the navigation objective. The more important evidence for this report is route coverage, the shape and continuity of the trajectory, the command profiles and the qualitative perception outputs.

5. Results and Discussion

5.1 Full-course trajectory

The representative development traces accumulate approximately 28.3 m of odometry distance and cover the main road structure of the development course. The updated controller follows a broadly similar global route to the baseline while showing larger local deviations around difficult geometry and obstacle-handling regions. These deviations are consistent with the additional recovery, safety and bypass states introduced in the final controller. The trajectory plots are used primarily to illustrate the spatial behaviour of the robot during the valid

portion of each recording. They should not be interpreted as evidence of completion time, because the recording process may continue even after the robot has left the meaningful operating region.

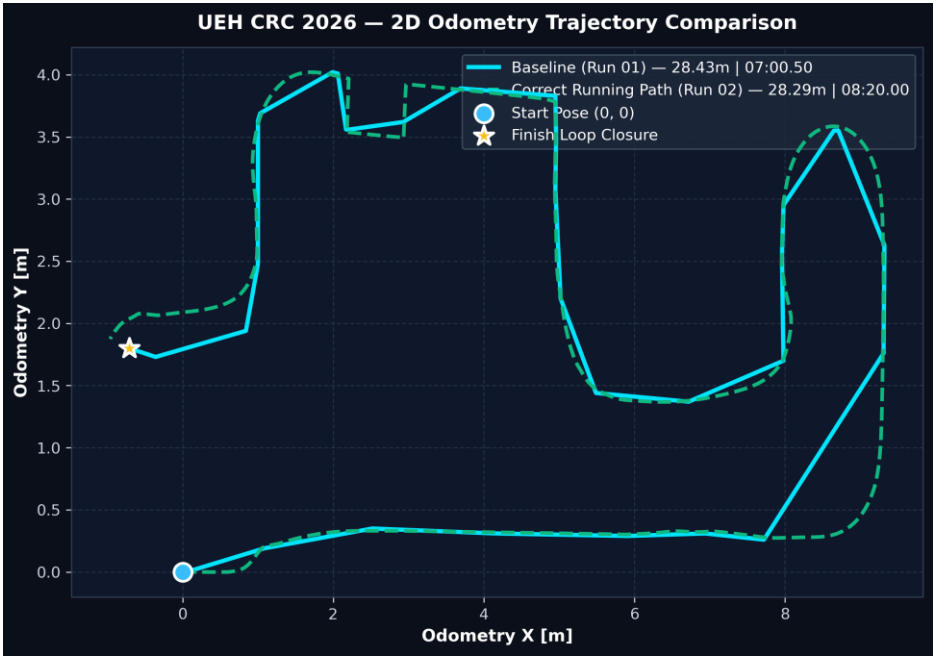


Figure 5. Odometry-frame trajectories for the representative recorded runs. This plot complements the top-down map by showing the path directly in the recorded odometry coordinate frame.

5.2 Cumulative distance progression

Cumulative odometry distance is used as a descriptive measure of how much motion was recorded during each development run. The baseline trace reaches approximately 28.430 m, while the updated trace reaches approximately 28.293 m. The values are therefore very similar in total accumulated odometry distance.

However, cumulative distance alone does not prove successful course completion, because odometry can continue to accumulate after a severe departure from the intended road or operating region. For this reason, distance is interpreted together with the trajectory plots and qualitative run observations rather than as an independent success criterion.

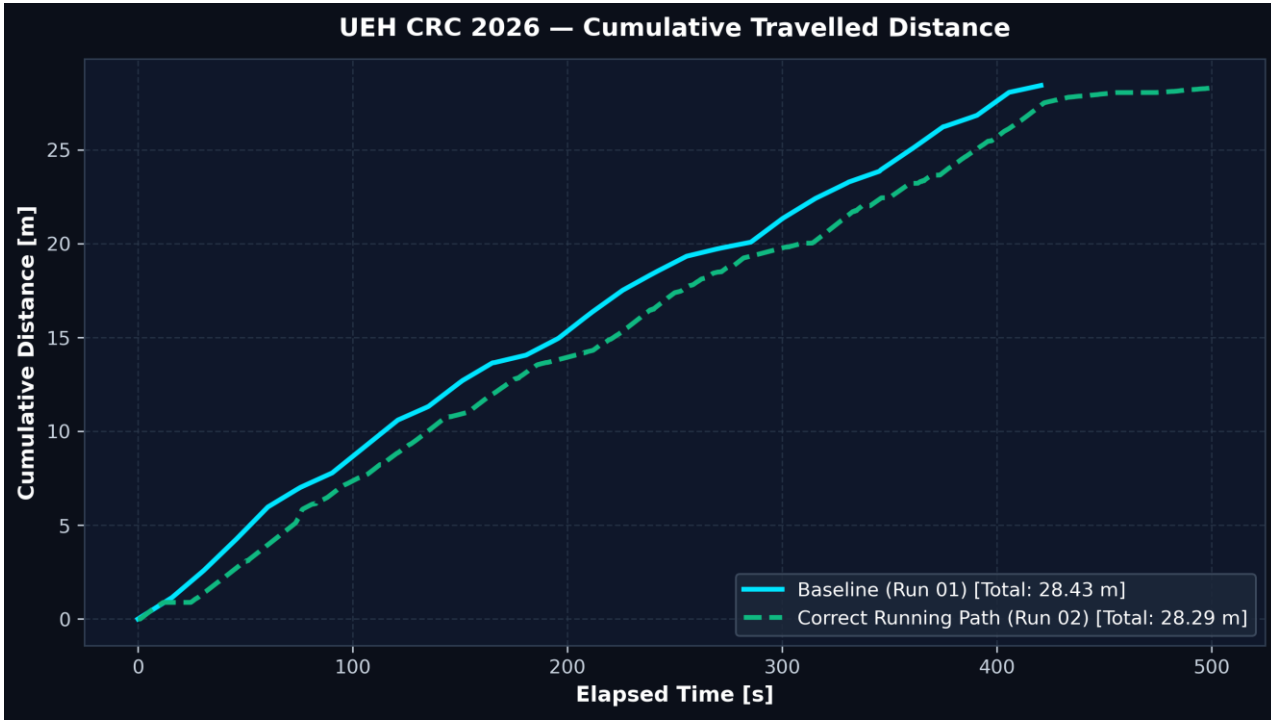


Figure 6. Cumulative travelled distance versus elapsed time for the recorded runs.

5.3 Linear-speed behaviour

The command logs show a clear change in forward-speed behaviour between the original baseline and the integrated controller. Mean linear command falls from approximately 0.099 m/s in the baseline trace to 0.0586 m/s in the updated trace, while the recorded maximum reaches 0.25 m/s.

The lower mean command is consistent with the more conservative state-dependent control policy of the final node, which includes reduced-speed steering, lane-recovery motion, safety holds and explicit stop states. The higher short-duration commands are also consistent with the implemented temporary speed modes, including the 0.25 m/s RAMP boost and the 0.20 m/s GREEN and crosswalk-clear behaviours.

The motion log alone does not identify which specific perception event produced each individual speed interval. Therefore, causal labels are not assigned retrospectively without synchronized perception-event logs.

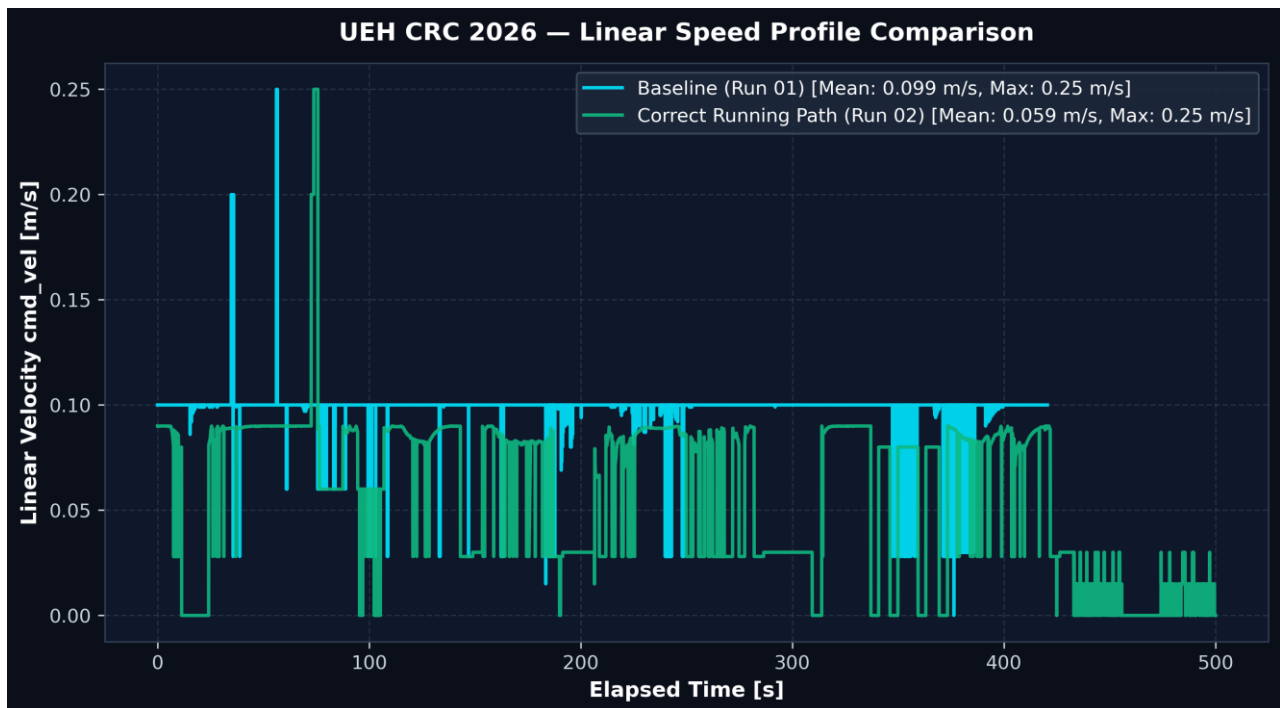


Figure 7. Linear-speed command profiles for the representative runs, showing the more conservative and state-dependent motion of the updated controller.

5.4 Angular activity and steering

The updated controller produces substantially more angular command activity than the original baseline trace. Mean absolute angular command in the updated run is approximately 0.0742 rad/s, compared with approximately 0.0015 rad/s in the baseline log, while the updated run reaches a maximum absolute angular command of approximately 0.455 rad/s.

This broader steering activity is consistent with the additional control mechanisms present in the final system, including connected-polyline tracking, staged lost-lane recovery, IMU-guided heading correction and structured obstacle-bypass manoeuvres. Angular-command magnitude should therefore be interpreted as a measure of controller activity rather than as a direct measure of lane-tracking accuracy.

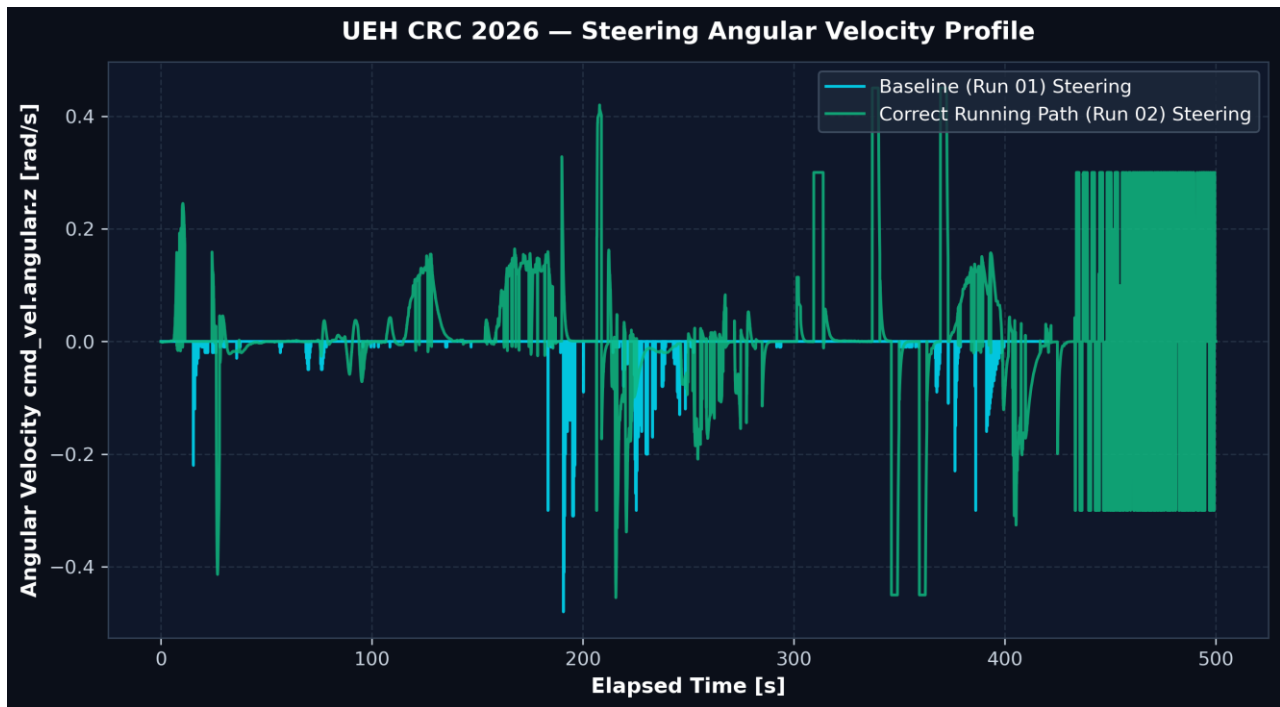


Figure 8. Absolute/angular velocity command profile illustrating steering effort and discrete recovery or bypass corrections.

5.5 Perception evidence

The perception composites provide qualitative evidence that the camera pipeline can detect STOP, RAMP, highway entry/exit, BUS, TUNNEL, pedestrian-crossing signs, traffic-light colours and the simulator pedestrian.

In the final controller, several of these observations are connected directly to behaviour states. STOP detection activates a timed stop hold; confirmed RED and YELLOW traffic-light states stop the robot while GREEN releases motion; RAMP detection activates a temporary speed increase; CROSSWALK detection initiates the delayed pedestrian-monitoring sequence; and HW_ENTRY/HW_EXIT control whether obstacle bypass is permitted. BUS and TUNNEL remain perception-only observations and do not directly modify motion.

The available development dataset does not include synchronized class-by-class ground-truth annotations or a confusion matrix. Consequently, this report does not claim a formal numerical perception-accuracy percentage.

The two debug streams also separate navigation and semantic perception. The lane debug view visualizes lane geometry, tracking and recovery state, LiDAR information and steering behaviour, whereas the detection debug view displays sign detections, raw traffic-light observations, pedestrian status and the current behaviour state. This separation prevents visualization overlays from affecting the image used by the lane-following controller.

5.6 Overall assessment

The recorded experiments demonstrate the behaviour of the integrated navigation and decision stack across the development environment. Compared with the original baseline, the final controller contains substantially richer arbitration logic: connected-boundary lane following, staged lane-loss recovery, default obstacle stop/wait behaviour, highway-gated obstacle bypass, STOP handling, traffic-light stop/go control, RAMP speed adaptation and crosswalk pedestrian handling.

The principal trade-off of this design is increased controller complexity. More behaviour states, confirmation timers and safety gates create a larger parameter space and increase the amount of steering and low-speed corrective motion observed in the command logs. This conservative behaviour is intentional and prioritizes recoverability and safety over aggressive forward motion.

Recorded run duration is not used as a performance metric in this analysis because the logger may continue operating after the robot has already left the meaningful driving region. Consequently, trajectory, cumulative

distance, command profiles and qualitative behaviour evidence are treated as the more informative development-level indicators.

6. Limitations and Future Work

The first limitation is the hand-engineered nature of the camera perception. HSV thresholds, right-side sign ROI constraints, contour geometry and symbol heuristics are lightweight and explainable, but they are tuned to the supplied simulator appearance and can be sensitive to illumination, camera exposure, scale and partial occlusion. BUS and TUNNEL are intentionally detection-only classes, while the behaviour-coupled classes rely on temporal confirmation to reduce false activations.

Second, several behaviours are controlled by fixed timing and threshold parameters. STOP uses a 1.0 s confirmation and 2.0 s hold, traffic-light decisions require confidence above 0.86 for 0.8 s, CROSSWALK uses a 0.5 s sign confirmation followed by a 3.0 s delay, and highway entry uses a short continuous confirmation before bypass is armed. These choices are practical for the development track but should be stress-tested over wider start poses, lighting and randomized scene timing before being treated as universally optimal.

Third, lane quality is evaluated internally in image-pixel space rather than with an independent physical centreline error measurement. The current recorded logs therefore support trajectory, distance and command-profile analysis, but not an official lane RMS score. A stronger evaluation package would add an offline centreline evaluator and synchronized behaviour-event logs so that stops, boosts, bypasses and recovery episodes can be matched directly to the figures without changing the controller itself.

Fourth, the obstacle bypass contains a deliberate open-loop geometric component. Lateral and forward legs are executed from odometry distance while heading is regulated from IMU/odometry yaw; camera perception is intentionally paused until the detour ends. This makes the manoeuvre deterministic and avoids false sign/light triggers during the pass, but it can accumulate geometric error if wheel slip, obstacle placement or lane width differs significantly from the development scenario.

Finally, some deadlock-avoidance safeguards are development-oriented. Traffic-light handling can auto-release after an extended 10 s stop, and the signal interaction is also marked complete once an odometry-x guard is passed. These mechanisms prevent indefinite waiting in the supplied simulation but may be sensitive to a changed start pose or altered signal placement. The quantitative comparison also uses only a small number of representative unique traces; more randomized-seed and start-pose runs would provide stronger evidence of repeatability, especially for rare recovery and behaviour-arbitration transitions.

7. AI Usage Statement

AI assistance was used during development as a coding and writing aid. It was used to propose implementation ideas, explain ROS/OpenCV errors, suggest refactoring and debugging steps, and help organize technical documentation. AI assistance was also used to draft and revise parts of this report from the participant-provided code, run logs and figures.

The submitted solution was not treated as automatically accepted AI output. Proposed changes were integrated into the ROS 2 node, executed in the supplied simulator and retained or revised based on observed behaviour. Parameter choices, final integration decisions, simulation runs, result collection and submission decisions remained under participant control. The report distinguishes BUS/TUNNEL detection-only functionality from the behaviour-coupled STOP, traffic-light, RAMP, CROSSWALK/pedestrian and highway-entry/exit state machines.

No external trained model file is required by the submitted controller. The final implementation uses ROS 2, OpenCV, NumPy and the interfaces already available in the provided simulation environment.

8. Conclusion

The final V4.8 solution demonstrates a practical, CPU-friendly autonomy stack for the UEH CRC 2026 simulation environment. Its navigation core is a connected-boundary lane representation with adaptive thickness filtering, temporal tracking and preview-aware steering. Around that core, the final node adds staged IMU/odometry recovery, LiDAR safety, a highway-gated obstacle bypass, STOP holds, traffic-light stop/go, RAMP speed control and crosswalk pedestrian arbitration without relying on external trained model weights.

The updated controller covers approximately the same cumulative development-route distance as the baseline while using lower average forward speed and more active steering. This reflects the selected trade-off: robustness, temporal confirmation, recovery and safety arbitration are prioritised over minimum elapsed time. The current evidence therefore emphasizes route coverage, trajectory continuity, command profiles and qualitative perception examples rather than treating raw completion time as the main success criterion.

The remaining work is focused on validation rather than missing core behaviours: quantify lane RMS with an offline evaluator, synchronize event logs with motion traces, stress-test the threshold/timing choices over varied start poses and random seeds, and replace route-sensitive development safeguards where necessary. The existing architecture and recorded evidence provide a strong basis for those tests and for later migration toward the hardware round.

References

- [1] UEH Creative Robot Contest 2026, Simulation Round Participant Handbook, organizer-provided documentation.
- [2] Final submitted controller source: `src/crc_sim/crc_sim/starter_node.py`, V4.8 (connected right-boundary polyline, behaviour-coupled sign/light/crosswalk logic, highway-gated bypass and staged recovery).
- [3] Participant-side run records: `run_01_baseline.csv`, `run_02.csv`, corresponding metadata, run summaries and analysis figures.
- [4] Repository instructions: `README.md` and `VIDEO.md` supplied with the submission package.